
Documento Técnico Final

Sistema ciberfísico:
3 en raya con Ned-2

Autor: Pablo Navarro Domínguez

Fecha: 16 de mayo de 2026

Repositorio: https://github.com/pnavarro3/Proy_SistCiber

Índice

1. Resumen	2
2. Alcance del documento final	2
3. Descripción técnica de la solución final	3
3.1. Arquitectura general del sistema	3
3.2. Subsistemas software desarrollados	4
3.3. Características principales y decisiones de diseño por subsistema	5
4. Validación de requisitos	10
4.1. Requisitos verificados previamente	10
4.2. Verificación de requisitos del subsistema	11
5. Guía rápida para la puesta en marcha	17
5.1. Prerrequisitos	17
5.2. Secuencia de arranque recomendada	17
6. Conclusiones	18
A. Anexo A: Contenido del repositorio	19
A.1. Estructura general	19
A.2. Descripción por carpetas	19
A.3. Archivos clave para operación del sistema	20
A.4. Acceso al repositorio	20

1 Resumen

Este documento técnico final consolida el desarrollo del sistema ciberfísico de 3 en raya con Ned-2, integrando los avances recogidos en los dos documentos técnicos previos y alineándolos con el enunciado del proyecto. El objetivo funcional del sistema es ejecutar una partida física completa frente a un rival humano, combinando supervisión en tiempo real, manipulación segura, detección visual del tablero y comunicación remota entre nodos de control.

Para facilitar la revisión académica y el acceso al material entregable, el repositorio del proyecto está disponible en: https://github.com/pnavarro3/Proy_SistCiber.

La solución se apoya en componentes concretos implementados y validados: un núcleo Python para control de partida, movimiento y visión en el robot, junto con bloques MATLAB/Simulink en el PC central para emisión y recepción de mensajes TCP/IP.

La implementación se estructura en dos capas de decisión y ejecución: el PC central, responsable del control supervisado mediante máquina de estados en Simulink, y el PC del robot, donde se ejecuta la lógica Python de movimiento, visión, inteligencia artificial y mensajería. El robot mantiene además una segunda conexión TCP/IP opcional hacia otro robot Ned-2 rival, lo que permite extender el escenario de juego a una partida máquina contra máquina en la que cada robot recibe los movimientos del contrario. Esta separación ha permitido mantener trazabilidad de eventos, reducir acoplamiento entre subsistemas y facilitar la verificación incremental por hitos.

En la versión final del proyecto el sistema dispone de funcionamiento operativo completo en ambos modos de juego: el modo manual, en el que el PC central emite los movimientos a ejecutar, y el modo automático, en el que el propio robot calcula su jugada mediante una heurística que prioriza la victoria inmediata y, en su defecto, la ocupación del centro o una posición aleatoria libre. El protocolo de mensajes TCP/IP cubre el ciclo completo de partida, incluyendo la emisión sistemática de los mensajes de cierre FIN VIC, FIN DER y FIN EMP en todos los caminos de terminación posibles. La detección visual del tablero, las secuencias de movimiento pick-and-place y el flujo de sincronización de turnos mediante la entrada digital DI1 se encuentran igualmente operativos.

2 Alcance del documento final

El alcance de este informe incluye la descripción técnica de la solución desarrollada en el PC central, el PC de control del robot, el subsistema de visión y la capa de comunicaciones. También cubre las decisiones de diseño adoptadas para satisfacer las restricciones del enunciado, en particular las asociadas a seguridad cinemática, sincronización de turnos y capacidad de supervisión remota.

Además, se incorporan detalles de hardware y software que ya aparecían en los dos documentos previos: la configuración de visión del Ned-2, la posición home como estado seguro de inicio, las limitaciones temporales de recogida y transporte, el uso de mensajes

de texto terminados en salto de línea y el papel de la IA en el modo automático.

Desde el punto de vista de verificación, el documento organiza el estado de cumplimiento de requisitos, indicando qué aspectos ya fueron validados en entregables anteriores y qué aspectos se validan en este documento. Se mantiene, además, una estructura de evidencias compatible con la trazabilidad exigida por el cliente.

Hito	Objetivo de proyecto	Cobertura documental actual	Estado
H1	Aprobación del documento de requisitos	Referencia de requisitos y matriz de validación	Completado
H2	Verificación del PC central	Arquitectura de control y protocolo Simulink	Completado
H3	Verificación del robot manipulador	Control de movimiento, visión y seguridad	Completado
H4	Validación del sistema ciberfísico	Procedimientos y evidencias de cierre	Completado

Tabla 1: Cobertura del documento respecto a los hitos del proyecto.

3 Descripción técnica de la solución final

3.1 Arquitectura general del sistema

La arquitectura final implementa un esquema distribuido de supervisión y ejecución. El PC central actúa como nodo de control de alto nivel y concentra la interfaz de usuario, la máquina de estados y la coordinación de la partida. El PC del robot ejecuta la capa de campo: control de movimiento, percepción visual, cálculo de jugada en modo automático y notificación de eventos al sistema central.

El ciclo operativo comienza con la selección de modo y la verificación de precondiciones de seguridad (home + Start), continúa con la alternancia de turnos y finaliza cuando se detecta una condición de victoria, derrota o empate. Esta estructura se diseñó para satisfacer las exigencias del enunciado sobre supervisión continua del estado y control remoto del robot.

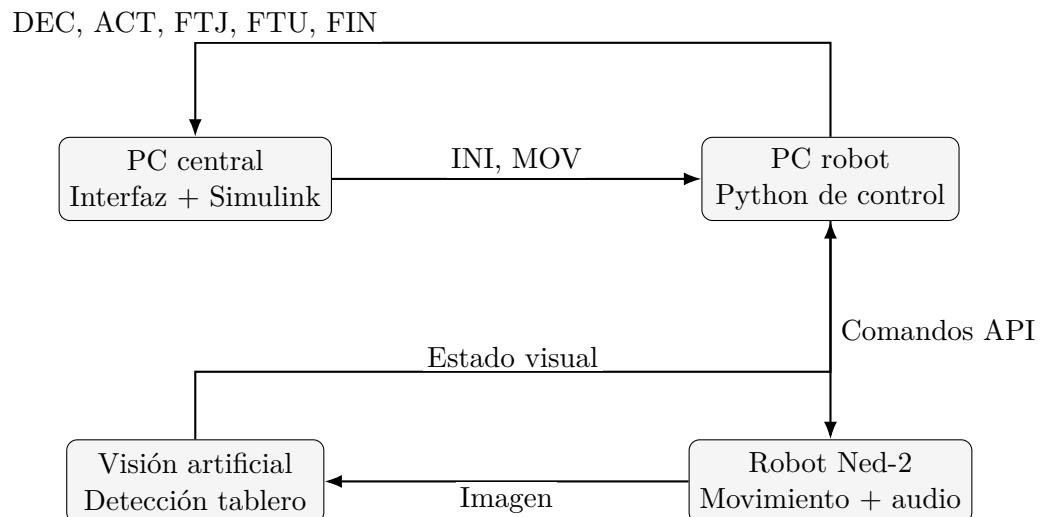


Figura 1: Gráfica de arquitectura funcional y flujo de información del sistema.

Subsistema	Tecnología principal	Función principal en el sistema
PC central	Matlab/Simulink	Supervisar estados, coordinar turnos y presentar estado de partida al usuario.
PC robot	Python + sockets	Ejecutar lógica de juego, control de movimiento y comunicación con el PC central.
Robot Ned-2	API pyniryo	Manipular piezas en tablero siguiendo restricciones cinemáticas de seguridad.
Visión artificial	OpenCV + cámara del robot	Detectar estado del tablero rival tras su turno y actualizar partida.

Tabla 2: Resumen de responsabilidades técnicas por subsistema.

3.2 Subsistemas software desarrollados

Los ficheros desarrollados para el PC del robot y el PC central permiten distinguir claramente las capas de responsabilidad del sistema. En Python, ComRobot.py concentra el bucle de partida, control de turnos, sockets y secuencias de movimiento; Ned2_ULL.py agrupa utilidades de configuración y persistencia de poses; y VisionRobot.py implementa la detección del estado del tablero. En Simulink/MATLAB, el intercambio de datos y su adaptación al modelo se materializan mediante bloques de emisión, recepción e interpretación de señales.

Archivo	Tipo	Responsabilidad principal
ComRobot.py	Python	Bucle principal de partida, secuenciación de movimientos, gestión de turnos y mensajes de estado.
Ned2_ULL.py	Python	Utilidades de persistencia de poses, restauración de configuración y apoyo al robot.
VisionRobot.py	Python	Segmentación visual del tablero y extracción de casillas ocupadas para actualizar el estado de juego.
SimulinkEmisor.m	MATLAB/Simulink	Formateo de comandos INI/MOV en tramas <code>uint8[1x32]</code> para el bloque TCP/IP Send.
SimulinkReceptor.m	MATLAB/Simulink	Recepción con buffer persistente, parseo de DEC/ACT/FTU/FTJ/FIN y generación de pulsos y estado de tablero.
InterpretarTablero.m	MATLAB/Simulink	Descomposición del vector de tablero 1x9 en nueve señales escalares para lógica de estados/visualización.
MensajeTurno.m	MATLAB/Simulink	Generación de mensaje ASCII de turno (Espera / Te toca jugar) para interfaz/supervisión.

Tabla 3: Relación de componentes software desarrollados y su función técnica.

3.3 Características principales y decisiones de diseño por sub-sistema

3.3.1 PC central (Simulink + control de partida)

El PC central implementa el control supervisado en tiempo real solicitado en el enunciado. La máquina de estados en Simulink secuencia las fases de espera, orden de movimiento, confirmación de ejecución y evaluación de continuidad de partida. Esta representación permite validar transiciones de forma explícita y facilita la interpretación del estado por parte del operador.

La interfaz de comunicación se realiza mediante funciones MATLAB que convierten comandos y eventos entre señales de simulación y tramas de protocolo. Se adoptó una semántica de pulsos de un ciclo para eventos discretos (DEC, FTJ, FTU, FIN), evitando disparos múltiples y mejorando el acoplamiento temporal con el modelo de estados.

Los bloques MATLAB incluidos en el repositorio cubren de forma explícita la frontera de comunicación con el robot. SimulinkEmisor.m genera siempre un vector fijo de 32 bytes de tipo `uint8`, lo que simplifica su conexión con el bloque TCP/IP Send de Simulink y evita variabilidad de tamaño en tiempo de ejecución. El bloque soporta mensajes **INI MAN**, **INI AUT**, **INI COM** y **MOV origen destino**, incluyendo la traducción de identificadores de almacén 11–13 a las cadenas `alm1–alm3`. También contempla un modo de emisión de **MOV** sin argumentos para escenarios de sincronización o prueba.

Por su parte, SimulinkReceptor.m implementa un buffer persistente de recepción,

acumula bytes procedentes del bloque TCP/IP Receive y procesa líneas completas terminadas en CR/LF. El diseño desacopla recepción y decodificación, permitiendo absorber fragmentación de red sin perder mensajes. Sobre ese buffer se reconocen los comandos DEC, ACT, FTU, FTJ y FIN. Para los eventos discretos se utilizan contadores persistentes que mantienen pulsos de 100 ms equivalentes en muestras, calculados a partir del periodo T_s del modelo. De esta forma, el resto de la máquina de estados recibe señales limpias y temporizadas.

En el caso del mensaje ACT, el bloque receptor transforma la cadena de nueve celdas del tablero en un vector numérico de longitud 9 apto para tratamiento en Simulink. Esta conversión permite que la lógica del PC central trabaje directamente con un estado discreto del tablero, sin necesidad de parseado textual adicional. El mensaje FIN actualiza además la salida RES con etiquetas de texto (**victoria**, **derrota** o **empate**), lo que simplifica la integración con estados finales y paneles de supervisión del modelo.

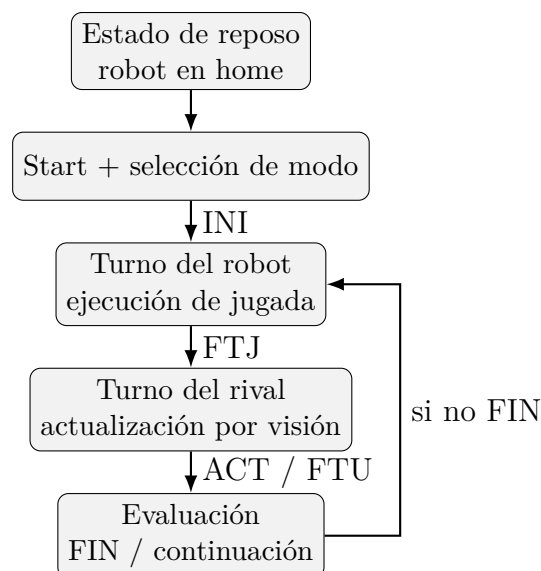


Figura 2: Gráfica de ciclo de turno en la máquina de estados del sistema.

Bloque Simulink	Interfaz	Comportamiento implementado
SimulinkEmisor.m	Salida uint8[1x32]	Genera tramas de longitud fija para INI y MOV, con terminador de línea y compatibilidad directa con TCP/IP Send.
SimulinkReceptor.m	Entradas data, count, Ts	Acumula bytes en buffer persistente, procesa mensajes por línea y expone pulsos discretos y estado de tablero.
SimulinkReceptor.m	Salidas DEC, FTU, FTJ, FIN, RES, tablero	Entrega eventos discretos temporizados y un vector de 9 posiciones para la lógica de control central.

Tabla 4: Resumen funcional del código MATLAB/Simulink incluido en el PC central.

3.3.2 Subsistema robot (control de movimiento)

El control de movimiento del Ned-2 está implementado en ComRobot.py, donde se integra la recepción de comandos, la planificación de secuencias pick-and-place y la notificación de estado. El sistema soporta modos manual y automático sobre una arquitectura común de turnos, lo que permite conservar el mismo ciclo de ejecución independientemente de quién decide la jugada. Además de la conexión principal con el PC central, el código contempla una segunda conexión TCP/IP opcional hacia otro robot Ned-2, lo que habilita un escenario de juego máquina contra máquina en el que cada robot actúa como rival del otro.

Se ha adoptado una estrategia de seguridad basada en desacoplo cinemático Z/X-Y, tal como exige el enunciado. El robot primero se posiciona en una pose de aproximación, desciende verticalmente para recoger/dejar, y retorna al plano de aproximación antes de cualquier desplazamiento horizontal. Esta lógica se complementa con limitación de velocidad (valor 50) y retorno a la pose de visión general para el cierre de ciclo. El fin de turno rival se dispara mediante entrada digital DI1, tratada como evento de pulsador físico.

Elemento del Ned-2	Dato técnico	Uso en el proyecto
Grados de libertad	6	Permiten posicionar el efector final sobre el tablero y en la zona de reserva.
Sistema de visión	Cámara RGB integrada	Captura el tablero para detectar movimientos del rival.
Efector final	Pinza de dedos	Recogida y colocación de fichas.
Conectividad del robot	TCP/IP / rosbridge	Acceso mediante API pyniryo desde Python.
Límite de velocidad	50 en la configuración usada	Garantizar movimientos suaves y seguros.

Tabla 5: Resumen del hardware y configuración funcional del Ned-2 empleada en el proyecto.

3.3.3 Sistema de visión

El subsistema de visión utiliza la cámara integrada en el robot para actualizar el estado del tablero tras el turno rival. El pipeline de procesamiento incluye normalización geométrica del workspace, segmentación por color en HSV y asignación de detecciones a la rejilla 3x3. El resultado se publica como posiciones discretas en el protocolo de comunicación.

Una decisión de diseño especialmente relevante ha sido separar fuentes de verdad: el estado de fichas propias se mantiene por lógica interna, mientras que el estado rival se obtiene por visión. Esta separación reduce sensibilidad a errores de detección de las propias piezas y mejora la consistencia del ciclo de decisión automática.

Durante el desarrollo se incorporó una herramienta interactiva de calibración HSV me-

diante trackbars para facilitar ajustes según condiciones de iluminación. Esta funcionalidad fue especialmente importante en las pruebas iniciales, ya que el documento de requisitos no solo exige detectar la partida sino hacerlo de forma fiable y repetible en campo.

3.3.4 Comunicación TCP/IP

La comunicación remota se ha implementado sobre TCP/IP con mensajes ASCII terminados en fin de línea. Esta elección prioriza simplicidad de depuración y compatibilidad directa entre Python y Simulink, manteniendo suficiente expresividad para incluir acción solicitada, estado de proceso y estado de partida, tal como especifica el enunciado.

En la dirección PC central → robot se transmiten órdenes de inicialización y movimiento; en la dirección opuesta se reportan eventos de decisión, actualización visual y cierre de turno. El receptor Simulink ya contempla mensaje FIN con resultado, pero su emisión en el bucle principal Python se considera tarea abierta de cierre funcional. En la dirección PC central → robot se transmiten órdenes de inicialización y movimiento; en la dirección opuesta se reportan eventos de decisión, actualización visual y cierre de turno. En la versión actual de `ComRobot.py`, los mensajes de cierre FIN VIC, FIN DER y FIN EMP se emiten de forma sistemática en todos los caminos de terminación de partida (victoria propia, derrota o empate), quedando alineados con el parser del receptor Simulink.

Estructura del mensaje	Contenido	Interpretación
IDN arg1 arg2 ... \n	Identificador + argumentos	Formato genérico legible y compatible con parser por línea.
INI modo \n	Modo = MAN / AUT / COM	Inicio de partida en el modo seleccionado.
MOV origen destino \n	Origen y destino	Orden de recogida y colocación en modo manual.
DEC \n	Evento discreto	Confirmación de jugada decidida en modo automático.
ACT cadena9 \n	Estado completo del tablero	Cadena de 9 celdas con símbolos _ / X / O.
FTJ / FTU \n	Evento de fin de turno	Cierre de turno del robot / del rival.
FIN res \n	VIC / DER / EMP	Cierre de partida emitido por el robot y consumido por Simulink para estado final.

Tabla 6: Formato general de los mensajes intercambiados entre PC central y robot.

Mensaje	Origen	Momento de uso
INI modo	PC central	Justo antes de comenzar la partida.
DEC	Robot	Cuando la IA ha calculado su movimiento.
ACT cadena9	Robot	Tras capturar la imagen al terminar el turno rival.
FTJ	Robot	Al finalizar el movimiento físico propio.
FTU	Robot	Al cerrar el turno del rival humano.
FIN res	Robot	Emitido al detectarse victoria, derrota o empate para cerrar la partida.

Tabla 7: Secuencia funcional de mensajes empleada durante una partida completa.

Dirección	Mensaje	Información funcional transportada
PC central → robot	INI modo	Inicio de partida y selección de modo de operación (MAN/AUT).
PC central → robot	MOV origen destino	Orden de recogida y colocación de ficha en modo manual.
Robot → PC central	DEC	Confirmación de decisión tomada en modo automático.
Robot → PC central	ACT cadena9	Estado completo del tablero con codificación _ / X / O.
Robot → PC central	FTJ / FTU	Fin de turno del robot / fin de turno del rival.
Robot → PC central	FIN res	Resultado final de partida (VIC, DER o EMP), emitido en el flujo operativo actual.

Tabla 8: Tabla de interfaz del protocolo de mensajería PC central–robot.

3.3.5 Mensajería de audio

La mensajería de audio forma parte del flujo activo de `ComRobot.py` mediante llamadas `robot.say` en hitos clave (inicio de partida, cesión de turno y resultado final), lo que aporta realimentación local al operador sin depender exclusivamente de la interfaz del PC central.

3.3.6 IA y decisión automática

La toma de decisión en modo automático implementada en `ComRobot.py` se concentra en la función `decidir_jugada(fichas_propias, fichas_rival)`. El algoritmo empieza calculando las casillas libres a partir de la diferencia entre las fichas propias, las del rival y el tablero completo. Con ese estado construye una lista de candidatos distinta según la fase de juego: si el robot todavía tiene menos de tres fichas en tablero, los movimientos se generan desde el almacén correspondiente (`alm1`, `alm2` o `alm3`) hacia cada casilla libre; cuando ya tiene tres fichas colocadas, los candidatos se obtienen moviendo cada ficha propia hacia cualquiera de las casillas libres.

Antes de elegir al azar, el código recorre todos esos candidatos y simula el resultado sobre una copia de las fichas propias para comprobar si la jugada produce victoria inmediata mediante `verificar_victoria`. Si existe al menos un movimiento ganador, ese movimiento se devuelve de forma directa. Si no hay victoria inmediata, el algoritmo aplica una regla adicional: en la primera jugada propia, si la casilla 5 está libre, se devuelve la jugada `alm1 5`; en el resto de situaciones, se selecciona una pareja `origen-destino` al azar entre todos los movimientos generados. Si no hay candidatos disponibles, la función devuelve `(None, None)` y el bucle principal interpreta ese caso como empate. El resultado es una IA muy simple, de coste lineal, pero con una preferencia clara por cerrar partida cuando ya existe una victoria posible.

4 Validación de requisitos

4.1 Requisitos verificados previamente

La tabla siguiente refleja el estado de los requisitos ya verificados en anteriores documentos técnicos.

RqId	Estado actual	Fuente de evidencia	Evidencia técnica observada
000C-003	Verificado	Subsistema de visión del robot	Captura de imagen comprimida, extracción workspace y segmentación HSV.
000C-025	Verificado	ComRobot.py + SimulinkReceptor.m	Comunicación remota TCP/IP y parseo de eventos discretos.
000C-026	Verificado	ComRobot.py	Intercambio PC-robot con sockets, mensajes TX/RX y terminador <code>\n</code> .
000C-030	Verificado	Inicialización del control del robot	Límite de velocidad de brazo fijado a 50 en la configuración de arranque.
000C-034	Verificado	ComRobot.py	Fin de turno rival por lectura de entrada digital DI1.
000C-038	Verificado	ComRobot.py	Mensajes codificados como ASCII/UTF-8 con salto de línea final.
000C-009	Verificado	ComRobot.py	Modo manual activo con recepción MOV, sin validaciones de robustez completas.
000C-010	Verificado	ComRobot.py	Modo automático activo con heurística aleatoria; no minimax en versión actual.
000C-036	Verificado	ComRobot.py	La función de victoria existe, pero no cierra el bucle principal de partida.
000C-037	Verificado	ComRobot.py	MAX_TURNOS definido, pendiente de aplicación efectiva en flujo principal.

4.2 Verificación de requisitos del subsistema

Se han realizado varias partidas completas para los modos de juego de las cuales se han elegido dos videos. Los enlaces dirigen al momento del video donde se muestra la verificación concreta de cada requisito.

4.2.1 Sistema de juego 3 en raya

RqId: [000C-001]

[Descripción del sistema de juego 3 en raya]

Verificación: Test - Ejecutar partidas completas y verificar cumplimiento de reglas en todos los modos habilitados.

Resultado: Se completan dos partidas: una en modo manual (<https://youtu.be/pjByXUHJplQ>) y otra en modo automático (<https://youtu.be/cVZLd5qQ-9E>).

4.2.2 Movimiento por turno

RqId: [000C-005]

Verificar que el robot solo se mueve en su turno y permanece en espera en turno rival.

Verificación: Test - Observar comportamiento del robot durante partidas de prueba alternando turnos.

Resultado: Hasta que no se toma la decisión en modo manual el robot no se mueve (<https://www.youtube.com/watch?v=pjByXUHJplQ&t=11s>) y hasta que no se pulsa el botón, no inicia el robot el movimiento en modo automático (<https://youtu.be/cVZLd5qQ-9E&t=48s>).

4.2.3 Piezas apiladas - validación en tablero

RqId: [000C-007]

Validar uso de pila mientras existan piezas y reubicación desde tablero al agotarse.

Verificación: Test - Ejecutar partida completa y registrar acceso a almacenes y traslados.

Resultado: En la partida en modo automático se prioriza siempre la pieza superior del almacén (<https://youtu.be/cVZLd5qQ-9E>). El código que lo demuestra es el siguiente:

```
1 if len(fichas_propias) < 3:
2     origen_alm = f"alm{len(fichas_propias) + 1}"
3     for libre in libres:
4         movimientos.append((origen_alm, str(libre)))
5 else:
6     for origen in fichas_propias:
7         for libre in libres:
8             movimientos.append((str(origen), str(libre)))
```

Listing 1: Generación de movimientos priorizando piezas del almacén.

4.2.4 Modos de juego

RqId: [000C-008]

Comprobar disponibilidad y selección de modos (manual, automático, competición) en interfaz y máquina de estados.

Verificación: Test - Verificar que los tres modos son seleccionables y funcionan de forma independiente.

Resultado: Los modos manual y automático están disponibles en el interfaz de Simulink (<https://youtu.be/cVZLd5qQ-9E&t=3s>). El modo competición está de manera experimental pero no implementado en el interfaz ya que se podrá jugar en manual o automático contra otro robot.

4.2.5 Modo manual

RqId: [000C-009]

Ejecutar batería de partidas completas en modo manual para cerrar validación formal de robustez y errores de entrada.

Verificación: Test de sistema - Realizar al menos 5 partidas completas en modo manual.

Resultado: Se han realizado 5 partidas en modo manual como por ejemplo la siguiente (<https://youtu.be/pjByXUHJp1Q>).

4.2.6 Modo automático

RqId: [000C-010]

Registrar decisiones automáticas y evaluar calidad de jugada de la heurística actual en campo.

Verificación: Análisis + Test - Ejecutar partidas en modo automático y registrar decisiones.

Resultado: Se han realizado 5 partidas en modo automático como por ejemplo la siguiente (<https://youtu.be/cVZLd5qQ-9E>).

4.2.7 Modo competición

RqId: [000C-011]

Ejecutar partida robot vs robot sin intervención manual.

Verificación: Test - Verificar funcionamiento del modo competición con dos robots.

Resultado: Queda pendiente de verificar de manera presencial.

4.2.8 Posición de inicio

RqId: [000C-012]

Confirmar permanencia del robot en home hasta que se pulse Start.

Verificación: Test - Verificar que el robot permanece en posición home antes de iniciar partida.

Resultado: El robot se mantiene en posición de home hasta que no se pulsa el botón de la interfaz que inicia la partida (<https://youtu.be/cVZLd5qQ-9E&t=5s>).

4.2.9 Selección jugador inicial

RqId: [000C-014]

Verificar selector y correspondencia con primer turno de la partida.

Verificación: Test - Ejecutar partidas iniciadas por ambos jugadores y verificar que comienza quien corresponde.

Resultado: No se ha implementado dado que siempre empieza el jugador/ordenador.

4.2.10 Cambio de modo restringido

RqId: [000C-015]

Probar cambio de modo en partida y fuera de partida; validar bloqueo/permitido según estado.

Verificación: Test - Intentar cambiar modo durante partida (debe bloquearse) y fuera de partida (debe permitirse).

Resultado: Una vez seleccionado el modo de juego no es posible cambiarlo (<https://youtu.be/cVZLd5qQ-9E&t=3s>).

4.2.11 Desacoplo cinemático Z vs X-Y

RqId: [000C-017]

Revisar trayectorias del robot durante recogida y colocación de piezas para verificar desacoplo cinemático.

Verificación: Test - Observar ejecución real: aproximación horizontal, descenso vertical, recogida, ascenso vertical, desplazamiento horizontal.

Resultado: Se establecen dos planos XY de movimiento horizontal entre los que solo hay desplazamientos puramente verticales (<https://youtu.be/cVZLd5qQ-9E&t=48s>).

4.2.12 Recogida de piezas

RqId: [000C-018]

Comprobar aproximación X-Y y descenso vertical para agarre de piezas.

Verificación: Test - Verificar secuencia de aproximación horizontal y descenso vertical durante recogida.

Resultado: Se establece una posición de aproximación al almacen de fichas desde la que se accede a las mismas de manera estrictamente vertical (<https://youtu.be/cVZLd5qQ-9E&t=80s>)

4.2.13 Ascenso tras recogida

RqId: [000C-019]

Verificar ascenso vertical tras agarre antes de cualquier desplazamiento horizontal.

Verificación: Test - Observar que el robot asciende verticalmente tras recoger antes de moverse horizontalmente.

Resultado: Se establecen posiciones de aproximación para todas las posiciones del tablero para que se ascienda verticalmente después del agarre de una ficha (<https://youtu.be/pjByXUHJp1Q&t=173s>).

4.2.14 Movimientos X-Y en plano de aproximación

RqId: [000C-020]

Confirmar que los desplazamientos horizontales solo se realizan en el plano de aproximación.

Verificación: Test - Verificar que los movimientos horizontales se ejecutan siempre en la altura de aproximación.

Resultado: Todos los puntos que se encuentran en el plano de aproximación están a la misma altura (<https://youtu.be/pjByXUHJp1Q&t=177s>)

4.2.15 Mensajes de audio

RqId: [000C-022]

Integrar audio en flujo principal y validar reproducción en estados clave.

Verificación: Test - Ejecutar partida y verificar mensajes de audio en: inicio, movimiento y fin de partida.

Resultado: Se han establecido mensajes de inicio de partida, turno del rival y final de partida (<https://youtu.be/cVZLd5qQ-9E&t=5s>).

4.2.16 Actualización por visión

RqId: [000C-023]

Contrastar tras cada turno el estado detectado con el tablero real.

Verificación: Test - Ejecutar partidas y verificar que la detección visual coincide con el estado real del tablero.

Resultado: Se reciben mensajes de actualización de pantalla después de cada turno jugado que se ven reflejado en el interfaz de Simulink (<https://youtu.be/pjByXUHJp1Q&t=150s>).
Nota: el tablero se representa desde el punto de vista del robot.

4.2.17 Interfaz de usuario PC central

RqId: [000C-024]

Verificar presencia y funcionamiento de elementos requeridos de interfaz en el PC central.

Verificación: Test - Verificar elementos de interfaz: selector de modo, botón Start, visualización de tablero, estado de partida.

Resultado: La interfaz incluye un botón de inicio de partida, un selector de modo de

juego, un display del estado de la partida, el tablero de juego actualizado, un display para determinar el turno en modo manual y selectores de posición de origen y destino junto con botón de toma de decisión (<https://youtu.be/pjByXUHJp1Q&t=195s>).

4.2.18 Estado del robot conocido

RqId: [000C-027]

Revisar trazas para comprobar disponibilidad continua del estado del robot.

Verificación: Análisis - Registrar trazas de comunicación y verificar que el estado es conocido en todo momento.

Resultado: Se dispone de un display que indica si el robot esta en movimiento o en el turno del rival ('Espera') o esperando a la decisión del jugador ('Te toca jugar') (<https://youtu.be/pjByXUHJp1Q&t=195s>).

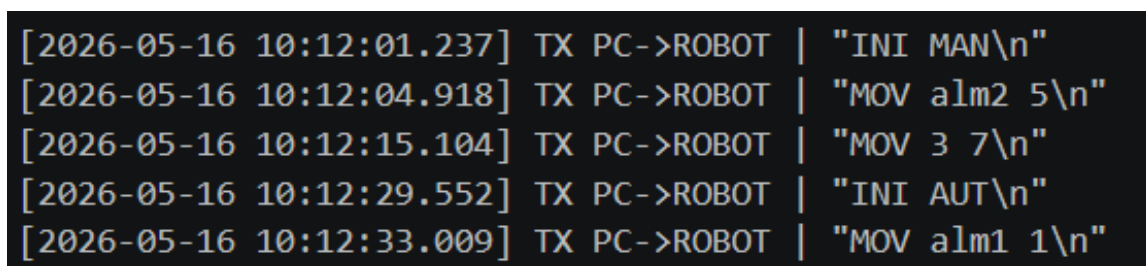
4.2.19 Formato de mensajes PC → robot

RqId: [000C-028]

Validar estructura y argumentos de mensajes de comando en trazas reales.

Verificación: Test - Capturar trazas de mensajes INI y MOV, verificar que cumplen formato especificado.

Resultado: Se capturaron trazas de emisión del PC central hacia el robot durante una secuencia de arranque y jugadas. Se verificó la presencia de comandos INI y MOV con formato correcto, argumentos válidos y terminación en salto de línea.



```
[2026-05-16 10:12:01.237] TX PC->ROBOT | "INI MAN\n"
[2026-05-16 10:12:04.918] TX PC->ROBOT | "MOV alm2 5\n"
[2026-05-16 10:12:15.104] TX PC->ROBOT | "MOV 3 7\n"
[2026-05-16 10:12:29.552] TX PC->ROBOT | "INI AUT\n"
[2026-05-16 10:12:33.009] TX PC->ROBOT | "MOV alm1 1\n"
```

Figura 3: Captura de trazas de comandos PC central → robot (RqId 000C-028).

4.2.20 Formato de mensajes robot → PC

RqId: [000C-029]

Validar estructura y argumentos de mensajes de estado/evento en trazas reales.

Verificación: Test - Capturar trazas de mensajes DEC, ACT, FTJ, FTU, FIN; verificar que cumplen formato especificado.

Resultado: Se capturaron trazas de retorno del robot al PC central durante el ciclo de turno completo. Las evidencias muestran recepción correcta de DEC, ACT, FTJ, FTU y FIN, con estructura conforme al protocolo y valores de tablero correctamente serializados.


```
[2026-05-16 10:12:05.441] RX ROBOT->PC | "DEC\n"
[2026-05-16 10:12:09.732] RX ROBOT->PC | "FTJ\n"
[2026-05-16 10:12:22.119] RX ROBOT->PC | "ACT 0 0 0 2 1 0 0 0 0\n"
[2026-05-16 10:12:22.401] RX ROBOT->PC | "FTU\n"
[2026-05-16 10:13:14.887] RX ROBOT->PC | "FIN VIC\n"
```

Figura 4: Captura de trazas de estado/evento robot $\rightarrow$ PC central (RqId 000C-029).

4.2.21 Tiempo máximo de recogida

RqId: [000C-031]

Medir al menos 5 ejecuciones de recogida y verificar que el tiempo es ≤ 15 segundos.

Verificación: Test - Ejecutar 5 recogidas consecutivas, cronometrar y registrar tiempos.

Resultado: Se realizan varias ciclos de medición de tiempos de recogida de piezas, en el siguiente video se puede ver que desde que recibe la orden ('1:50') hasta que agarra la pieza ('1:59') pasan 9 segundos (<https://youtu.be/pjByXUHJplQ&t=110s>).

4.2.22 Tiempo máximo de transporte

RqId: [000C-032]

Medir al menos 5 transportes completos y verificar que el tiempo es ≤ 15 segundos.

Verificación: Test - Ejecutar 5 ciclos de transporte completo, cronometrar y registrar tiempos.

Resultado: Se realizan varias ciclos de medición de tiempos desde que se agarra la pieza hasta que se suelta. en el siguiente video se puede ver que pasan 10 segundos desde que se agarra ('1:59') hasta que se suelta ('2:09') (<https://youtu.be/pjByXUHJplQ&t=119s>).

4.2.23 Tiempo máximo de decisión automática

RqId: [000C-033]

Medir tiempo de decisión en modo automático/competición y verificar que es ≤ 10 segundos.

Verificación: Test - Ejecutar partidas en modo automático, medir tiempos de decisión y registrar.

Resultado: La decisión en modo automático se toma de manera instantánea ya que cuando el rival pulsa el botón físico de final de turno el robot inicia su movimiento (<https://youtu.be/cVZLd5qQ-9E&t=48s>).

4.2.24 Fin de turno rival por botón físico

RqId: [000C-034]

Accionar botón físico (entrada digital DI1) y comprobar transición al siguiente estado.

Verificación: Test - Ejecutar batería repetida de accionamientos de botón físico, verificar que se detecta correctamente.

Resultado: El rival determina el final de su turno mediante la pulsación del botón físico conectado a la entrada digital 1 del robot (<https://youtu.be/cVZLd5qQ-9E&t=48s>).

5 Guía rápida para la puesta en marcha

5.1 Prerrequisitos

Hardware mínimo:

- Robot Ned-2 calibrado.
- Cámara del robot operativa.
- PC central y PC de control del robot conectados por red.
- Tablero y piezas en posición inicial.

Software mínimo:

- Entorno Python con dependencias del proyecto.
- Modelo Simulink con bloques de emisión/recepción configurados.
- Scripts de control de robot y visión disponibles.

5.2 Secuencia de arranque recomendada

1. Conectar el router del laboratorio y establecer conexión wifi del PC central
2. Encender el Ned-2 y conectar el PC del robot a su red wifi.
3. Revisar la IP asignada a cada equipo en la red del laboratorio.
4. Establecer la IP del PC del robot en los bloques TCP/IP Real-Time de Simulink (PC central).
5. Establecer la misma IP en el código del ordenador del robot (cliente/servidor Python según configuración).
6. Verificar conectividad de red entre PC central y PC del robot.
7. Calibrar el Ned-2.
8. Calibrar los valores HSV de la cámara del robot para la detección del tablero.
9. Cargar configuración de poses y parámetros de visión.
10. Iniciar servicios/scripts de control en el PC del robot.
11. Abrir el modelo de control en PC central (Simulink).

12. Configurar puertos y arrancar la comunicación.
13. Confirmar estado home y seleccionar modo de juego.
14. Pulsar Start para iniciar partida.

6 Conclusiones

El proyecto ha permitido diseñar, implementar y validar un sistema ciberfísico operativo para jugar al 3 en raya con robot Ned-2, integrando control supervisado en Simulink, ejecución de campo en Python, comunicación TCP/IP y percepción visual del tablero. La arquitectura distribuida adoptada (PC central + PC robot) ha resultado adecuada para separar responsabilidades, mantener trazabilidad de eventos y simplificar la verificación incremental por subsistemas.

Desde el punto de vista funcional, la solución implementa el ciclo completo de partida en modo manual y en modo automático, incluyendo secuenciación segura de movimientos, actualización del estado del tablero, notificación de turnos y cierre de partida con mensajes de resultado. La estrategia de desacoplo cinemático Z/X-Y, junto con la limitación de velocidad y el uso de pose de espera/home, ha permitido cumplir los objetivos de seguridad y robustez exigidos por el enunciado.

En validación, las evidencias recopiladas muestran cumplimiento de los requisitos principales de control remoto, sincronización de turnos, comunicación bidireccional y restricciones temporales de operación. En particular, las medidas de tiempos de recogida, transporte y decisión automática se mantienen dentro de los umbrales establecidos, y la interfaz del PC central proporciona información de estado continua para el operador.

Como limitaciones actuales, el modo de competición robot vs robot queda condicionado a validación presencial específica y la selección del jugador inicial no se ha priorizado en la implementación final. Estos puntos no comprometen el funcionamiento del escenario objetivo principal (robot vs humano), pero se identifican como líneas de cierre recomendadas para una versión posterior.

Como trabajo futuro, se propone: (i) completar la validación formal del modo competición en campo con dos robots, (ii) incorporar selección explícita de jugador inicial en la máquina de estados y (iii) reforzar la calibración asistida de visión para mejorar repetibilidad ante cambios de iluminación.

A Anexo A: Contenido del repositorio

Este anexo resume la organización del repositorio entregado, indicando la finalidad de cada carpeta y los archivos principales necesarios para reproducir, mantener y ampliar el sistema ciberfísico.

A.1 Estructura general

```

Proy_SistCiber/
+-- README.md
+-- .gitignore
+-- config/
|   +-- posbackup.json
+-- docs/
|   +-- Documento_tecnico_final.tex
|   +-- Documento_tecnico_final.pdf
+-- simulink/
|   +-- ProyectoSC.slx
|   +-- SimulinkEmisor.m
|   +-- SimulinkReceptor.m
|   +-- InterpretarTablero.m
|   +-- MensajeTurno.m
+-- src/
    +-- ComRobot.py
    +-- Ned2_ULL.py
    +-- VisionRobot.py
    +-- Calibracion.py

```

A.2 Descripción por carpetas

Ruta	Contenido y propósito
src/	Código Python del PC del robot: control de partida, comunicaciones TCP/IP, movimiento del Ned-2 y visión artificial.
simulink/	Modelo principal <code>ProyectoSC.slx</code> y funciones MATLAB/Simulink para emisión/recepción de mensajes e interpretación de tablero.
docs/	Documento técnico final de entrega: fuente LaTeX (<code>.tex</code>) y versión compilada (<code>.pdf</code>).
config/	Ficheros de configuración persistente del robot, en particular copia de seguridad de poses.

Tabla 10: Resumen funcional de carpetas del repositorio.

A.3 Archivos clave para operación del sistema

Archivo	Tipo	Rol en el sistema
src/ComRobot.py	Python	Bucle principal de juego en el robot, comunicación con PC central, ejecución de jugadas y mensajes de estado.
src/Ned2_ULL.py	Python	Utilidades de soporte del Ned-2 (poses, backup/restore y calibración HSV).
src/VisionRobot.py	Python	Detección visual del tablero y extracción de ocupación de casillas.
src/Calibracion.py	Python	Ajuste y calibración visual del sistema en entorno de desarrollo.
simulink/SimulinkEmisor.m	MATLAB	Construcción de tramas INI/MOV para transmisión TCP/IP.
simulink/SimulinkReceptor.m	MATLAB	Decodificación de mensajes DEC/ACT/FTU/FTJ/FIN y señales de control.
simulink/ProyectoSC.slx	Simulink	Modelo principal de supervisión y control en el PC central.
docs/ Documento_tecnico_final.tex	LaTeX	Fuente del documento técnico final.
docs/ Documento_tecnico_final.pdf	PDF	Versión compilada para entrega y revisión.
config/posbackup.json	JSON	Persistencia de poses de referencia del robot.

Tabla 11: Archivos principales para ejecución y mantenimiento.

A.4 Acceso al repositorio

Para consulta del código fuente, historial de cambios y documentación asociada, el profesor puede acceder directamente al repositorio oficial del proyecto en:

https://github.com/pnavarro3/Proy_SistCiber